

## Лабораторна робота № 7-8

**Мета:** Набути практичних навичок у застосуванні методів автоматизованого тестування програмного забезпечення на різних рівнях — юніт-тестування, інтеграційного та системного тестування. Ознайомитися з принципами та інструментами розробки через тестування. На парактиці навчитися створювати, виконувати й аналізувати результати автоматизованих тестів для забезпечення якості ПЗ. Вивчити на практиці TDD — Test-Driven Development (розробка через тестування) як підхід до розробки ПЗ, що сприяє підвищенню якості коду та зниженню кількості дефектів.

### ЗАВДАННЯ:

#### ЗАВДАННЯ 1 Створення Unit-тестів

- У проєкті створити клас відповідно до варіанту (Додаток А). Зберегти клас у теці /Services.
- Створити unit-тести відповідно до вимог для тестування класу
- Виконати перевірку тестів

#### ЗАВДАННЯ 2 Створення Unit-тестів з mock-об'єктами

- Обрати один із сервісів, створених у попередніх лабораторних роботах (ЛР4–ЛР6). Ознайомитися з логікою сервісу та його методами.
- Додати методи, які виконують певні обчислення або агрегації над колекціями даних. Можна використати LINQ для реалізації цих методів. Додати валідацію вхідних даних і обробку можливих винятків.
- Створити окремий тестовий проєкт
- Реалізувати unit-тести
- Виконати перевірку тестів

#### ЗАВДАННЯ 3 Test-Driven Development

- Обрати один з варіантів сервісів з Додатку А - ваш варіант + 5 (наприклад, якщо ваш варіант 10, оберіть сервіс з номером 15). Розробіть UML Statechart Diagram життєвого циклу об'єкта.
- Застосувати TDD-підхід (Red → Green → Refactor)

#### ЗАВДАННЯ 4 Розробити інтеграційні тести

- Обрати один із контролерів, створених у попередніх лабораторних роботах (ЛР4–ЛР6). Ознайомитися з HTTP-методами контролерами.

|           |       |             |        |      |                                     |       |      |         |
|-----------|-------|-------------|--------|------|-------------------------------------|-------|------|---------|
|           |       |             |        |      | ЛР.ОК24.ПІ231.03.07-08              |       |      |         |
| Змін.     | Аркуш | № докум.    | Підпис | Дата | Автоматизоване тестування<br><br>ПЗ | Лім.  | Арк. | Акрушів |
| Розроб.   |       | Башук О. Ю. |        |      |                                     | Н     | 1    | 28      |
| Перевір.  |       | Жереб Д. В. |        |      |                                     | Х П К |      |         |
|           |       |             |        |      |                                     |       |      |         |
| Н. Контр. |       |             |        |      |                                     |       |      |         |
| Затверд.  |       |             |        |      |                                     |       |      |         |

- Для кожного HTTP-методу контролера створити окремий тестовий метод із зрозумілою назвою, що відображає суть тесту.
- Виконати перевірку тестів

### Хід роботи:

#### Завдання 1

Створено клас StringUtils із методами для написання для нього тестів:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Text;
using System.Threading.Tasks;
namespace Task1.Services
{
    public class StringUtils
    { public string Resource { get; private set; }
      public StringUtils(string reosource)
      {
          if (string.IsNullOrEmpty(reosource))
              throw new ArgumentNullException();
          this.Resource = reosource;
      }
      public StringUtils() : this("Test") { }

      public string Reverse()
      {
          string reverce = "";
          for(int i = Resource.Length - 1; i >= 0; i--)
              reverce += Resource[i];
          return reverce;
      }
      public bool IsPalindrom()
      {
          for(int i = 0; i < Resource.Length; i++)
```

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
|       |      |          |        |      |                        | 2    |
| Змін. | Арк. | № докум. | Підпис | Дата |                        |      |

```

        if (Resource[i] != Resource[Resource.Length - 1 - i])
            return false;
        return true;
    }

    public int CountWord()
    {
        int count = 0;
        string find = Resource.Trim();
        for (int i = 0; i < find.Length; i++)
            if (find[i] == ' ')
                count++;
        return count + 1;
    }

    public static int MaxLen(params string[] rows)
    {
        int maxLen = 0;
        if (rows is null || rows.Length == 0)
            throw new ArgumentNullException();
        foreach (var r in rows)
            if (r.Length > maxLen)
                maxLen = r.Length;
        return maxLen;
    }

    public char[] ToCharArray()
    {
        char[] array = new char[Resource.Length];
        for (int i = 0; i < Resource.Length; i++)
            array[i] = Resource[i];
        return array;
    }

    public string AddToEnd(string add)
    {
        if (add is null || string.IsNullOrEmpty(add))
            throw new ArgumentNullException();
        return Resource + add;
    }

```

```

    }
}
}

```

Написано тести для перевірки функціональності роботи класу:

```

using System.Collections;
using Task1.Services;

namespace Tests
{
    public class StrUtilsTest
    {

        public class ReverseTestData: IEnumerable<object[]>
        {
            public IEnumerator<object[]> GetEnumerator()
            {
                yield return new object[] { "Привіт", "тівирП" };
                yield return new object[] { "a", "a" };
                yield return new object[] { "12345", "54321" };
                yield return new object[] { "Hi", "iH" };
                yield return new object[] { "AbC", "CbA" };
            }

            IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
        }

        public static IEnumerable<object[]> TestContainsMethod()
        {
            yield return new object[] { "Кит", 'т' };
            yield return new object[] { "Два слова", ' ' };
            yield return new object[] { "_", '_' };
            yield return new object[] { "Велика буква", 'В' };
            yield return new object[] { "Word", 'd' };
        }

        [Fact]
        public void ArgumentNullExceptionTest()
        {

```

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
| Змін. | Арк. | № докум. | Підпис | Дата |                        | 4    |

```

    StringUtils stringUtils = new StringUtils();
    Assert.Throws<ArgumentNullException>(() => stringUtils.AddToEnd(""));
}

[Fact]
public void TypeTest()
{
    StringUtils stringUtils = new StringUtils("Слово");
    var type = stringUtils.CountWord();
    Assert.IsType<Int32>(type);
}

[Fact]
public void RangeTest()
{
    int maxLen = StringUtils.MaxLen("123", "1234", "12", "12345", "1");
    Assert.InRange(maxLen, 1, 10);
}

[Fact]
public void SameTest()
{
    StringUtils s0 = new StringUtils("Слово");
    StringUtils s1 = new StringUtils("Слово");
    Assert.NotSame(s0, s1);
}

[Theory]
[InlineData("око")]
[InlineData("ОкО")]
[InlineData("Око")]
[InlineData("О ко")]
[InlineData("Я несу гусеня")]
[InlineData("кит на морі романтик")]
[InlineData("Кит на морі романтик")]
public void PalindromTest(string palindrom)
{
    StringUtils stringUtils = new StringUtils(palindrom);
    bool isPalindrom = stringUtils.IsPalindrom();

```

```

    Assert.True(isPalindrom);
}

[Theory]
[ClassData(typeof(ReverseTestData))]
public void ReverseTest(string word, string expectedReverse)
{
    StringUtils stringUtilities = new StringUtils(word);
    string reverse = stringUtilities.Reverse();
    Assert.Equal(reverse, expectedReverse);
}

[Theory]
[MemberData(nameof(TestContainsMethod))]
public void CollectionTest(string word, char symbol)
{
    StringUtils stringUtils = new StringUtils(word);
    char[] chars = stringUtils.ToCharArray();
    Assert.Contains(symbol, chars);
}
}
}

```

Запущено тести на виконання:

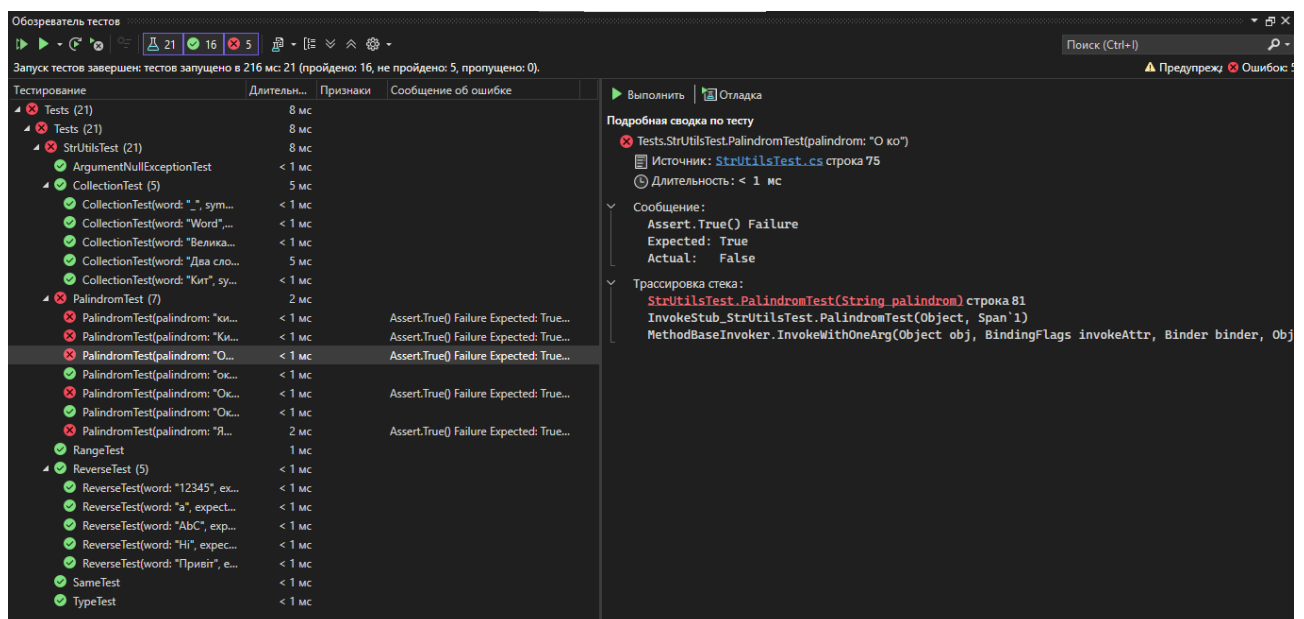


Рисунок 1 – Запуск тестів

Вияснено помилку, виправлено та запущено тести знову

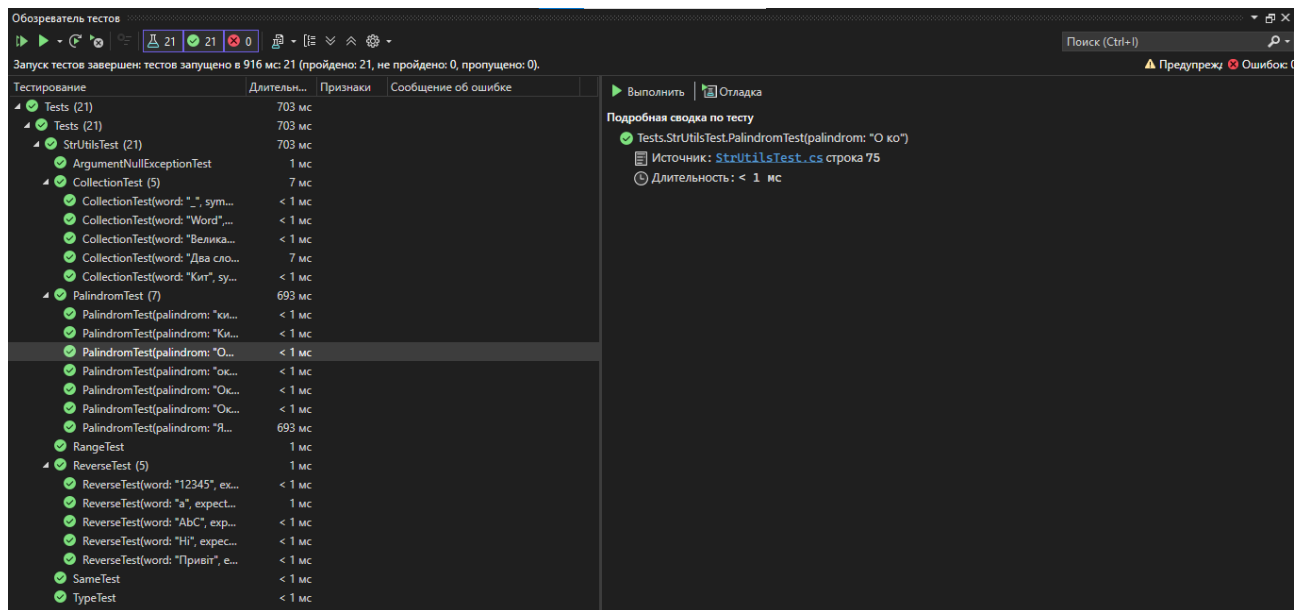


Рисунок 2 – Виправлено помилку та запущено тести знову

## Завдання 2

Додано агрегативні методи до сервісу для керування ресурсом підписок:

```
using LW4_task_3.Clients;
using LW4_task_3.Interface.Interfaces;
using LW4_task_3.Interface.InterfacesRepository;
using LW4_task_3.Models.Entities;
using MongoDB.Driver;
using System.Xml.Linq;

namespace LW4_task_3.Services
{
    public class SubService : ISubService
    {
        private readonly ISubRepository _subRepository;
        private readonly IPeopleRepository _peopleRepository;

        public SubService(ISubRepository subRepository, IPeopleRepository peopleRepository)
        {
            _subRepository = subRepository;
            _peopleRepository = peopleRepository;
        }

        public async Task CreateAsync(SubscriptionItem element)
        {

```

|       |      |          |        |      |                        |           |
|-------|------|----------|--------|------|------------------------|-----------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк.<br>7 |
| Змін. | Арк. | № докум. | Підпис | Дата |                        |           |

```

        await ValidOwner(element.OwnerId);
        await _subRepository.CreateAsync(element);
    }
    public async Task DeleteAsync(string id)
    {
        if (!await _subRepository.IsExist(id))
            throw new KeyNotFoundException($"Не знайдено підписки за вказаним Id {id}");
        await _subRepository.DeleteAsync(id);
    }
    public async Task<SubscriptionItem> GetByIdAsync(string id)
    {
        var sub = await _subRepository.GetByIdAsync(id);
        if (sub is null)
            throw new KeyNotFoundException($"Не знайдено підписки за вказаним Id {id}");
        return sub;
    }
    public async Task<IEnumerable<SubscriptionItem>> GetSubscriptionsItemsAsync(string?
ownerId, string? service, SubStatus? subStatus)
    {
        var subs = await _subRepository.GetSubscriptionsItemsAsync(ownerId, service, subStatu
s);
        if (subs is null || !subs.Any())
            throw new KeyNotFoundException("Не знайдено жодної підписки");
        return subs;
    }
    public async Task UpdateAsync(string id, SubscriptionItem element)
    {
        if (!await _subRepository.IsExist(id))
            throw new KeyNotFoundException($"Не знайдено підписки за вказаним Id {id}");
        await ValidOwner(element.OwnerId);
        element.Id = id;
        await _subRepository.UpdateAsync(id, element);
    }
    public async Task ValidOwner(string ownerId)

```

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
|       |      |          |        |      |                        | 8    |
| Змін. | Арк. | № докум. | Підпис | Дата |                        |      |



```

    {
        if (!await _peopleRepository.IsExist(ownerID))
            throw new ArgumentException($"Не знайдено власника з id {ownerID} для вказано
ї підписки");
    }

    public async Task<int> CountAllSub()
    {
        IEnumerable<SubscriptionItem> items;

        items = await _subRepository.GetSubscriptionsItemsAsync(null, null, null);    retur
n items.Count();
    }

    public async Task<double> PercentService(string ServiceName)
    {
        if (string.IsNullOrEmpty(ServiceName))
            throw new ArgumentNullException("Передано пустий параметр");

        IEnumerable<SubscriptionItem> items;

        items = await _subRepository.GetSubscriptionsItemsAsync(null, null, null);    int c
ountService = 0;

        if (!items.Any())
            return 0;

        foreach (var sub in items)
            if (sub.Service.ToLower() == ServiceName.ToLower())
                countService++;

        return ((double)countService / items.Count()) * 100;
    }

    public async Task<SubStatus> PopularStatus()
    {
        Dictionary<SubStatus, int> statuses = new Dictionary<SubStatus, int>();

        IEnumerable<SubscriptionItem> items = await _subRepository.GetSubscriptionsItemsA
sync(null, null, null);

        if (!items.Any())
            return SubStatus.None;

        foreach (var sub in items)
        {
            if (statuses.ContainsKey(sub.Status))

```

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
|       |      |          |        |      |                        | 9    |
| Змін. | Арк. | № докум. | Підпис | Дата |                        |      |

```

        statuses[sub.Status]++;
    else
        statuses.Add(sub.Status, 1);
    }
    return statuses.OrderByDescending(v => v.Value).First().Key;
}

public async Task<int> CountSubByStatus(SubStatus status)
{
    if (status == SubStatus.None || status == null)
        throw new ArgumentNullException("Невірний статус");
    IEnumerable<SubscriptionItem> items;
    items = await _subRepository.GetSubscriptionsItemsAsync(null, null, status);
    if (!items.Any())
        return 0;
    return items.Count();
}

public async Task<string[]> Top3Service()
{
    Dictionary<string, int> services = new Dictionary<string, int>();
    IEnumerable<SubscriptionItem> items = await _subRepository.GetSubscriptionsItemsA
sync(null, null, null);
    if (!items.Any())
        return Array.Empty<string>();
    foreach (var sub in items)
    {
        if (services.ContainsKey(sub.Service))
            services[sub.Service]++;
        else
            services.Add(sub.Service, 1);
    }
    return services.OrderByDescending(s => s.Value).Take(3).Select(s => s.Key).ToArray()
;
}
}

```

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
|       |      |          |        |      |                        | 10   |
| Змін. | Арк. | № докум. | Підпис | Дата |                        |      |

```
}
```

Розроблено тести для тестування функціоналу сервісу:

```
using LW4_task_3;
using LW4_task_3.Interface.InterfacesRepository;
using LW4_task_3.Models.Entities;
using LW4_task_3.Services;
using Microsoft.AspNetCore.Mvc.ModelBinding.Validation;
using Microsoft.AspNetCore.SignalR;
using Moq;
using System.Xml.Linq;
using ZstdSharp.Unsafe;

namespace SubServiceTest
{
    public class SubServiceTest
    {
        private readonly Mock<ISubRepository> _subRepositoryMock;
        private readonly Mock<IPeopleRepository> _peopleRepository;
        private readonly SubService _subService;

        private List<SubscriptionItem> GetSubscriptions()
        {
            return new List<SubscriptionItem>
            {
                new SubscriptionItem() { Id = "1", OwnerId = "1", Service = "Netflix", Status = Sub
Status.Active },
                new SubscriptionItem() { Id = "2", OwnerId = "1", Service = "Xbox", Status = SubS
tatus.Active },
                new SubscriptionItem() { Id = "3", OwnerId = "1", Service = "Amazon", Status = Su
bStatus.Active },
                new SubscriptionItem() { Id = "4", OwnerId = "2", Service = "Steam", Status = Sub
Status.Active },
                new SubscriptionItem() { Id = "5", OwnerId = "2", Service = "Google", Status = Su
bStatus.Overdue },
                new SubscriptionItem() { Id = "6", OwnerId = "2", Service = "Google", Status = Su
bStatus.Active },
                new SubscriptionItem() { Id = "7", OwnerId = "3", Service = "Google", Status = Su
bStatus.Archived },
            }
        }
    }
}
```

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
| Змін. | Арк. | № докум. | Підпис | Дата |                        | 11   |

```

        new SubscriptionItem() { Id = "8", OwnerId = "3", Service = "Netflix", Status = Sub
Status.Active },

        new SubscriptionItem() { Id = "9", OwnerId = "4", Service = "Google", Status = Su
bStatus.Active },

        new SubscriptionItem() { Id = "10", OwnerId = "4", Service = "Netflix", Status = Su
bStatus.Active },

        new SubscriptionItem() { Id = "11", OwnerId = "4", Service = "Megogo", Status = S
ubStatus.Archived },

        new SubscriptionItem() { Id = "12", OwnerId = "4", Service = "Amazon", Status = S
ubStatus.Active },

        new SubscriptionItem() { Id = "13", OwnerId = "5", Service = "AppleService", Statu
s = SubStatus.Active },

        new SubscriptionItem() { Id = "14", OwnerId = "5", Service = "Xbox", Status = Sub
Status.Active },

        new SubscriptionItem() { Id = "15", OwnerId = "5", Service = "Amazon", Status = S
ubStatus.Active }

    };
}

public SubServiceTest()
{
    _subRepositoryMock = new Mock<ISubRepository>();
    _peopleRepository = new Mock<IPeopleRepository>();
    _subService = new SubService(_subRepositoryMock.Object, _peopleRepository.Object);
}

[Fact]
public async Task CountAllSub_ReturnCount()
{
    var elements = GetSubscriptions();

    _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(null, null, null)).Returns
Async(elements);

    int count = await _subService.CountAllSub();

    Assert.Equal(elements.Count, count);
}

[Fact]
public async Task CountAllSub_Return0()
{

```

```

        _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(null, null, null)).Returns
Async(Enumerable.Empty<SubscriptionItem>);

        int count = await _subService.CountAllSub();

        Assert.Equal(0, count);
    }

    [Theory]
    [InlineData("Xbox", ((double)2 / 15) * 100, 0.001)]
    [InlineData("Netflix", ((double)3 / 15) * 100, 0.001)]
    [InlineData("Anonim", 0, 0.001)]

    public async Task PercentService_WhenCollectionIsNotEmpty(string name, double percent,
double normal)
    {
        var elements = GetSubscriptions();

        _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(null, null, null)).Returns
Async(elements);

        double percentService = await _subService.PercentService(name);

        Assert.Equal(percent, percentService, normal);
    }

    [Fact]

    public async Task PercentService_WhenCollectionIsEmpty()
    {
        _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(null, null, null)).Returns
Async(Enumerable.Empty<SubscriptionItem>);

        double percent = await _subService.PercentService("Netflix");

        Assert.Equal(0, percent);
    }

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData(null)]

    public async Task PercentService_ArgumentNullException(string? name)
    {
        await Assert.ThrowsAsync<ArgumentOutOfRangeException>(() => _subService.PercentService
(name));
    }

```

```

[Fact]
public async Task PopularStatus_NotEmpty()
{
    var element = GetSubscriptions();
    _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(null, null, null)).Returns
Async(element);
    var status = await _subService.PopularStatus();
    Assert.Equal(SubStatus.Active, status);
}

[Fact]
public async Task PopularStatus_CollectionEmpty()
{
    _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(null, null, null)).Returns
Async(Enumerable.Empty<SubscriptionItem>);
    var status = await _subService.PopularStatus();
    Assert.Equal(SubStatus.None, status);
}

[Theory]
[InlineData(SubStatus.Active, 12)]
[InlineData(SubStatus.Overdue, 1)]
[InlineData(SubStatus.Archived, 2)]
public async Task CountSubByStatus_TestReturn(SubStatus subStatus, int count)
{
    var elements = GetSubscriptions();
    _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(null, null, subStatus)).
ReturnsAsync(elements.Where(s => s.Status == subStatus));
    int c = await _subService.CountSubByStatus(subStatus);
    Assert.Equal(count, c);
}

[Theory]
[InlineData(SubStatus.None)]
[InlineData(null)]
public async Task CountSubByStatus_InvalidStatus(SubStatus subStatus)
{

```

```

        await Assert.ThrowsAsync<ArgumentNullException>(() => _subService.CountSubByStatus(subStatus));
    }

    [Fact]
    public async Task CountSubByStatus_EmptyCollection()
    {
        _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(null, null, SubStatus.Active)).ReturnsAsync(Enumerable.Empty<SubscriptionItem>);

        int count = await _subService.CountSubByStatus(SubStatus.Active);

        Assert.Equal(0, count);
    }

    [Fact]
    public async Task Top3Service_NotEmptyCollection()
    {
        var elements = GetSubscriptions();

        _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(null, null, null)).ReturnsAsync(elements);

        var services = await _subService.Top3Service();

        Assert.Equal(services, new string[] { "Google", "Netflix", "Amazon" });
    }

    [Fact]
    public async Task Top3Service_EmptyCollection()
    {
        _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(null, null, null)).ReturnsAsync(Enumerable.Empty<SubscriptionItem>);

        var services = await _subService.Top3Service();

        Assert.Empty(services);
    }

    [Fact]
    public async Task Create_ValidOwner()
    {
        var sub = new SubscriptionItem { Id = "New", OwnerId = "Valid", Service = "Netflix", Status = SubStatus.Active };

        _peopleRepository.Setup(r => r.IsExist(sub.OwnerId)).ReturnsAsync(true);

        await _subService.CreateAsync(sub);
    }

```

```

        _subRepositoryMock.Verify(r => r.CreateAsync(sub), Times.Once);
        _peopleRepository.Verify(r => r.IsExist(sub.OwnerId), Times.Once);
    }
    [Fact]
    public async Task Create_InvalidOwner_ArgumentException()
    {
        var sub = new SubscriptionItem { Id = "New", OwnerId = "Invalid", Service = "Netflix",
Status = SubStatus.Active };
        _peopleRepository.Setup(r => r.IsExist(sub.OwnerId)).ReturnsAsync(false);
        await Assert.ThrowsAsync<ArgumentException>(() => _subService.CreateAsync(sub));
        _subRepositoryMock.Verify(r => r.CreateAsync(sub), Times.Never);
        _peopleRepository.Verify(r => r.IsExist(sub.OwnerId), Times.Once);
    }
    // _peopleRepository.Setup(r => r.IsExist(sub.Id)).ReturnsAsync(true);
    [Fact]
    public async Task GetById_Exist()
    {
        var sub = GetSubscriptions()[1];
        _subRepositoryMock.Setup(r => r.GetByIdAsync(sub.Id)).ReturnsAsync(sub);
        var subFrom = await _subService.GetByIdAsync(sub.Id);
        Assert.Equal(sub, subFrom);
        _subRepositoryMock.Verify(r => r.GetByIdAsync(sub.Id), Times.Once);
    }
    [Fact]
    public async Task GetById_NotFound_KeyNorFoundException()
    {
        string id = "SomeId";
        _subRepositoryMock.Setup(r => r.GetByIdAsync(id)).ReturnsAsync((SubscriptionItem)
null!);
        await Assert.ThrowsAsync<KeyNotFoundException>(() => _subService.GetByIdAsync(
id));
    }
    public static IEnumerable<object[]> GetSubs() =>
        new List<object[]>

```



```

{
    new object[] { null, null, null },
    new object[] { "1", null, null },
    new object[] { null, "Netflix", null },
    new object[] { null, null, SubStatus.Active },
    new object[] { "1", null, SubStatus.Active },
    new object[] { null, "Netflix", SubStatus.Active },
    new object[] { "1", "Netflix", SubStatus.Active },
    new object[] { "1", "Netflix", SubStatus.Active }
};

[Theory]
[MemberData(nameof(GetSubs))]

public async Task GetSubscriptionsItems_ValidDate(string? ownerId, string? service, SubS
tatus? subStatus)
{
    var elements = GetSubscriptions();
    if (ownerId is not null)
        elements = elements.Where(x => x.Id == ownerId).ToList();
    if (service is not null)
        elements = elements.Where(x => x.Service == service).ToList();
    if (subStatus is not null)
        elements = elements.Where(x => x.Status == subStatus).ToList();
    _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(ownerId, service, subSt
atus)).ReturnsAsync(elements);
    var subs = await _subService.GetSubscriptionsItemsAsync(ownerId, service, subStatus);
    Assert.Equal(elements, subs);
}

[Theory]
[MemberData(nameof(GetSubs))]

public async Task GetSubscriptionsItems_NotFound(string? ownerId, string? service, SubSt
atus? subStatus)
{
    _subRepositoryMock.Setup(r => r.GetSubscriptionsItemsAsync(ownerId, service, subSta
tus)).ReturnsAsync(Enumerable.Empty<SubscriptionItem>());

```

```

        await Assert.ThrowsAsync<KeyNotFoundException>(() => _subService.GetSubscriptionsItemsAsync(ownerId, service, subStatus));
    }

    [Fact]
    public async Task Update_ValidOwnerAndExistSub()
    {
        string id = "UpdateId";

        var sub = new SubscriptionItem { Id = id, OwnerId = "ValidOwner", Service = "Service", Status = SubStatus.Active };

        _subRepositoryMock.Setup(r => r.IsExist(id)).ReturnsAsync(true);
        _peopleRepository.Setup(r => r.IsExist(sub.OwnerId)).ReturnsAsync(true);
        await _subService.UpdateAsync(id, sub);
        _subRepositoryMock.Verify(r => r.IsExist(id), Times.Once);
        _subRepositoryMock.Verify(r => r.UpdateAsync(id, sub), Times.Once);
        _peopleRepository.Verify(r => r.IsExist(sub.OwnerId), Times.Once);
    }

    [Fact]
    public async Task Update_NotFoundSub()
    {
        string id = "Invalid";

        _subRepositoryMock.Setup(r => r.IsExist(id)).ReturnsAsync(false);

        await Assert.ThrowsAsync<KeyNotFoundException>(() => _subService.UpdateAsync(id, new SubscriptionItem()));
    }

    [Fact]
    public async Task Update_InvalidOwner()
    {
        string id = "Valid";

        var sub = new SubscriptionItem() { OwnerId = "Invalid" };

        _subRepositoryMock.Setup(r => r.IsExist(id)).ReturnsAsync(true);
        _peopleRepository.Setup(r => r.IsExist(sub.Id)).ReturnsAsync(false);

        await Assert.ThrowsAsync<ArgumentException>(() => _subService.UpdateAsync(id, sub));
    }

    [Fact]

```

```

public async Task Delete_IdExit()
{
    string id = "Valid";
    _subRepositoryMock.Setup(r => r.IsExist(id)).ReturnsAsync(true);
    await _subService.DeleteAsync(id);
    _subRepositoryMock.Verify(r => r.DeleteAsync(id), Times.Once);
}

[Fact]
public async Task Delete_IdDontExist()
{
    string id = "Invalid";
    _subRepositoryMock.Setup(r => r.IsExist(id)).ReturnsAsync(false);
    await Assert.ThrowsAsync<KeyNotFoundException>(() => _subService.DeleteAsync(i
d));
}
}
}
}

```

Запущено тести на перевірку роботи

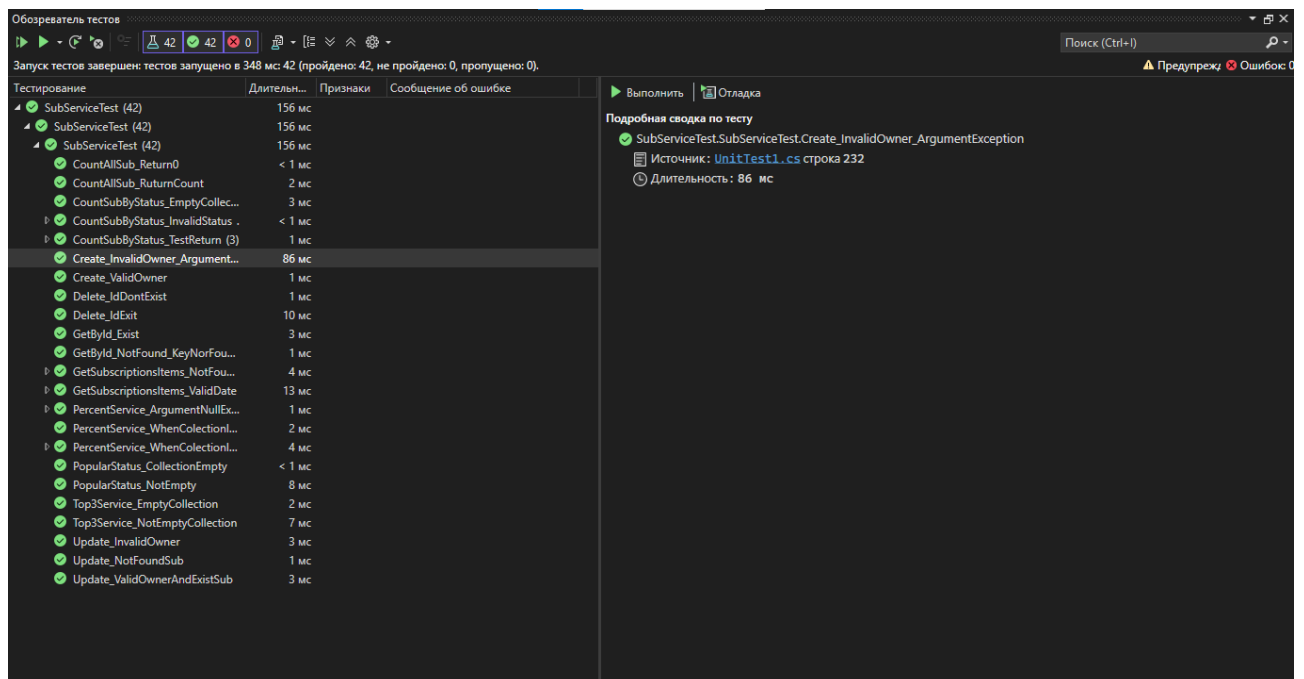


Рисунок 3 – Результат виконання тестів

### Завдання 3

Створено початковий клас EmailValidator із першим методом Ж

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
| Змін. | Арк. | № докум. | Підпис | Дата |                        | 19   |

```

namespace Task3.Services
{
    public static class EmailValidator
    {
        public static bool ValidEmail()
        {
            throw new NotImplementedException();
        }
    }
}

```

Створено тести для подальшої побудови методів класу:

```

using Task3.Services;
namespace EmailValidatorTest
{
    public class UnitTest1
    {
        public static IEnumerable<object[]> ValidEmails()
        {
            return new List<object[]>
            {
                new object[] { "testEmail@example.com" },
                new object[] { "user.Tim+2025@sub.domain.com" },
                new object[] { "12345@numeric-gmail.com" }
            };
        }

        public static IEnumerable<object[]> InvalidEmails()
        {
            return new List<object[]>
            {
                new object[] { "email" },
                new object[] { "email@gmail" },
                new object[] { "emailgmail.com" },
                new object[] { "emailGmail." },
                new object[] { "email@.com" },
            };
        }
    }
}

```

```

        new object[] { "email.com" },
        new object[] { " " },
        new object[] { "" }
    };
}

[Theory]
[MemberData(nameof(ValidEmails))]
public void ValidEmail(string email)
{
    var valid = EmailValidator.IsValidEmail(email);
    Assert.True(valid);
}

[Theory]
[MemberData(nameof(InvalidEmails))]
public void InvalidEmail(string email)
{
    var valid = EmailValidator.IsValidEmail(email);
    Assert.False(valid);
}
}
}

```

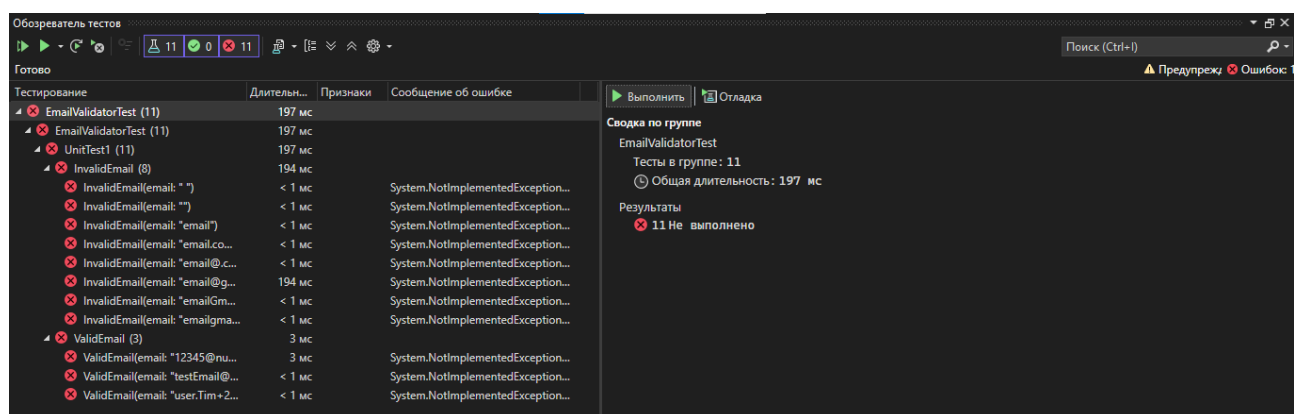


Рисунок 4 – Запущено тести, отримано помилки

Надано мінімальну реалізацію для виконання тестів:

```

public static class EmailValidator
{
    public static bool IsValidEmail(string email)
    {

```

```

    return Regex.IsMatch(email,@"^[^\s@]+@[^\s@]+\.[^\s@]{2,}$");
}
}

```

Запущено тести для перевірки роботи:

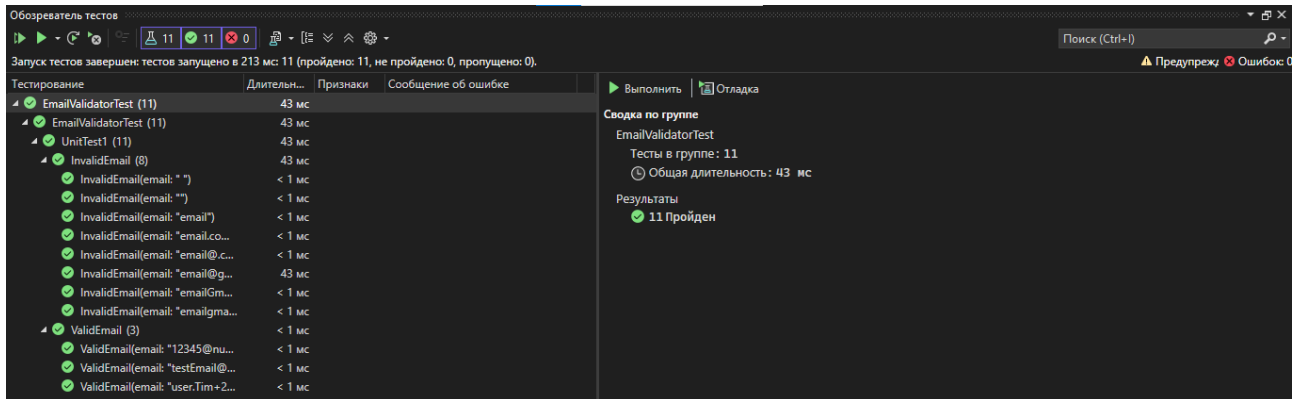


Рисунок 5 – Пройдені тести після мінімальної реалізації

Здійснено рефакторинг методу:

namespace Task3.Services

```

{
    public static class EmailValidator
    {
        public const string Standart = @"^[^\s@]+@[^\s@]+\.[^\s@]{2,}$";

        public static bool IsValidEmail(string email)
        {
            if(string.IsNullOrEmpty(email))
                return false;

            return Regex.IsMatch(email,Standart);
        }
    }
}

```

Запущено знову тести:

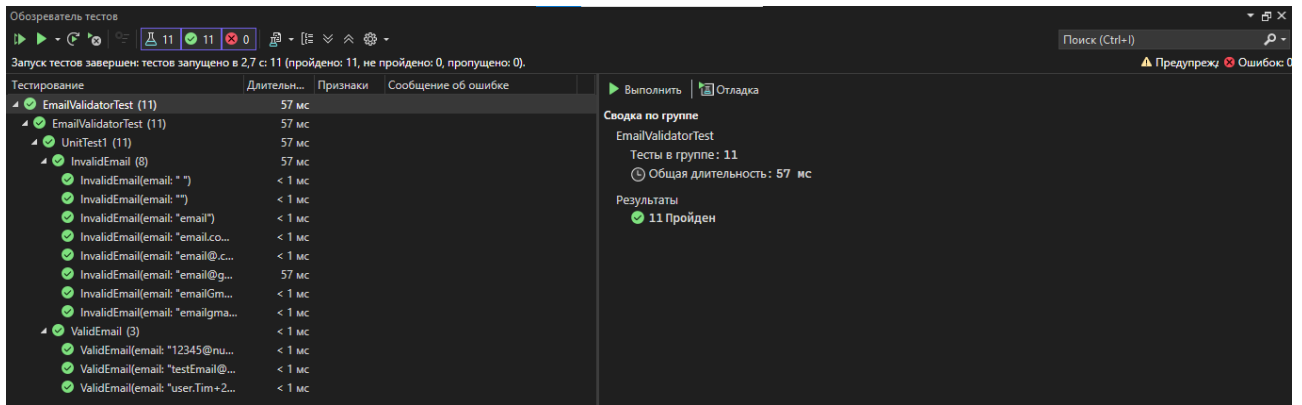


Рисунок 6 – Результат запуску тестів після рефакторингу

Створено новий метод із пустою реалізацією:

```
public static string GetDomain(string email)
```

```
{
    throw new NotImplementedException();
}
```

Реалізовано тести для нового методу:

```
public static IEnumerable<object[]> Domains()
```

```
{
    return new List<object[]>
    {
        new object[] { "email@gmail.com", "gmail.com" },
        new object[] { "email@Gmail.com", "gmail.com" },
        new object[] { "email@Gmail.COM", "gmail.com" },
        new object[] { "email@some-domain.net", "some-domain.net"},
        new object[] { "email@some-domain.ua.com", "some-domain.ua.com"}
    };
}
```

```
public static IEnumerable<object[]> InvalidDomains()
```

```
{
    return new List<object[]>
    {
        new object[] { "" },
        new object[] { " " },
        new object[] { null },
        new object[] { "email@.com" },
        new object[] { "email.com" },
    };
}
```

```

};
}
[Theory]
[MemberData(nameof(Domains))]
public void Domain(string email, string domain)
{
    string dom = EmailValidator.GetDomain(email);
    Assert.Equal(domain, dom);
}
[Theory]
[MemberData(nameof(InvalidDomains))]
[MemberData(nameof(InvalidEmails))]
public void InvalidDomain(string email)
{
    Assert.Throws<FormatException>(() => EmailValidator.GetDomain(email));
}

```

Запущено тести на виконання:

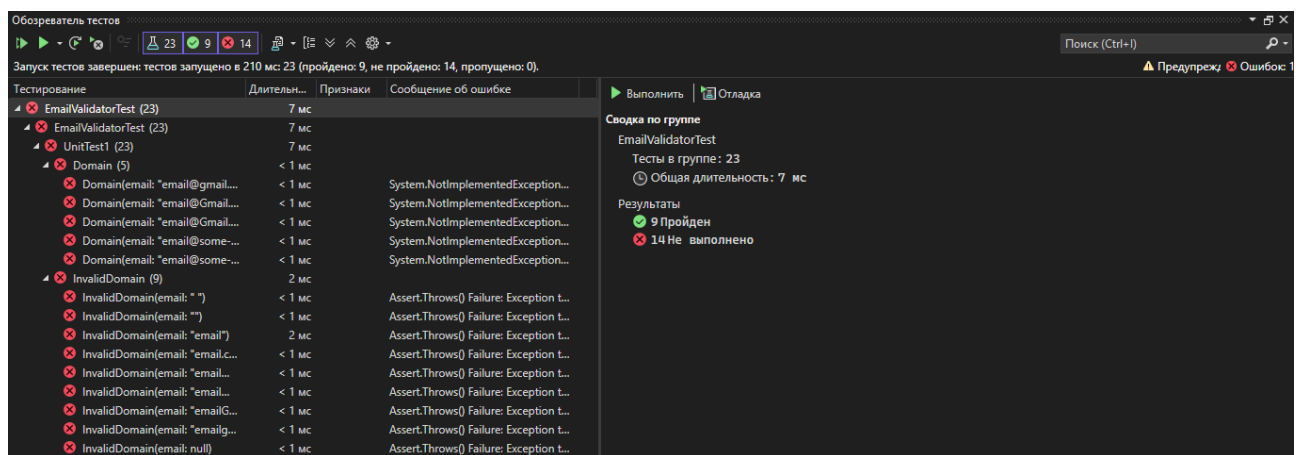


Рисунок 7 – Результат запуску тестів

Надано мінімальну реалізацію для проходження тестів:

```

public static string GetDomain(string email)
{
    if (string.IsNullOrEmpty(email) || !IsValidEmail(email))
        throw new FormatException();
    int index = 0;
    for (int i = 0; i < email.Length; i++)
        if (email[i] == '@')
            index = i;
}

```

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
|       |      |          |        |      |                        | 24   |
| Змін. | Арк. | № докум. | Підпис | Дата |                        |      |



```

return email.Substring(index + 1).ToLower();
}

```

Запущено тести на перевірку реалізації:

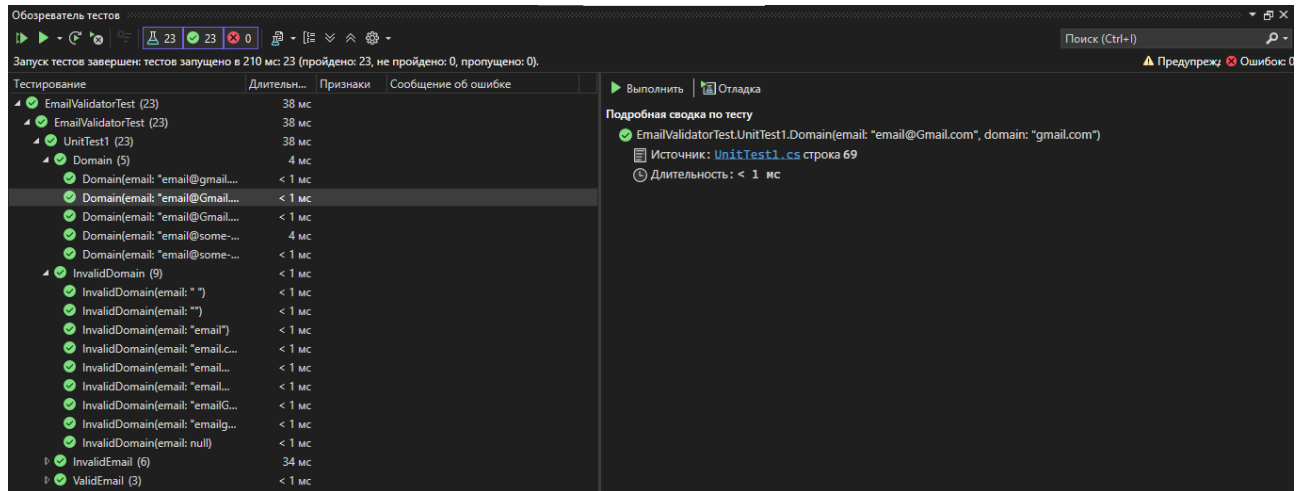


Рисунок 8 – Результат запуску тестів

Здійснено рефакторинг методу:

```

public static string GetDomain(string email)
{
    if (string.IsNullOrEmpty(email) || !IsValidEmail(email))
    {
        throw new FormatException("Невірний формат електронної адреси");
    }
    int index = email.IndexOf('@');
    return email.Substring(index + 1).ToLower();
}

```

Знову запущено тести для перевірки роботи:

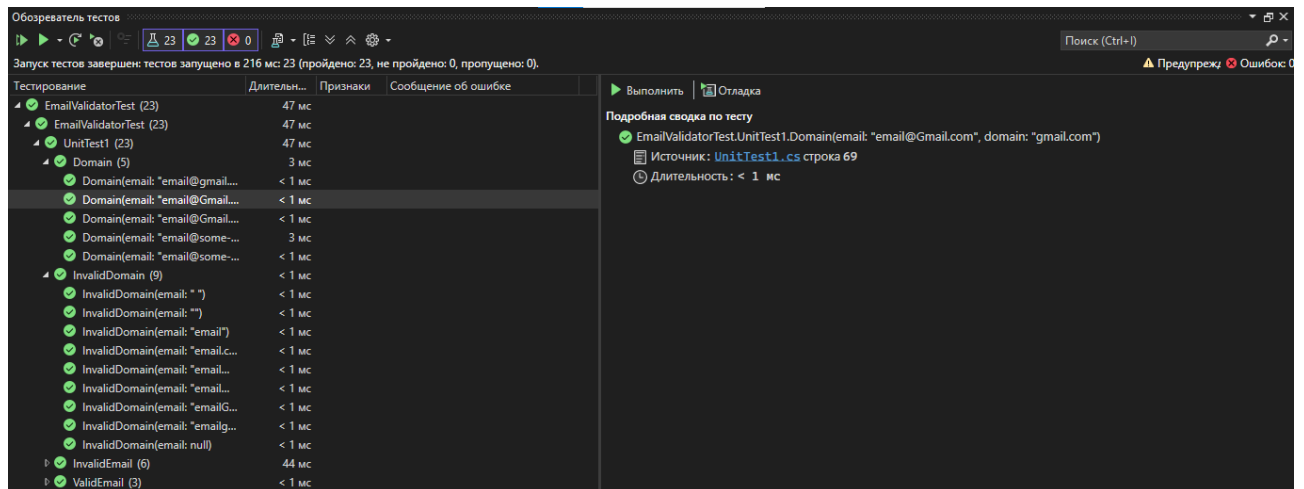


Рисунок 9 – Запуск тестів після рефакторингу

Створено ще один метод для реалізації функціоналу класу:

```

public static string SetNormalGmail(string email)

```

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
|       |      |          |        |      |                        | 25   |
| Змін. | Арк. | № докум. | Підпис | Дата |                        |      |

```

{
    throw new NotImplementedException();
}

```

Розроблено тести для вказаного методу:

```

public static IEnumerable<object[]> SetNormalGmailEmails()
{
    return new List<object[]>
    {
        new object[] { "testEmail@example.com", "testEmail@gmail.com" },
        new object[] { "user.Tim+2025@sub.domain.com", "userTim@gmail.com" },
        new object[] { "12345@numeric-gmail.com", "12345@gmail.com" },
        new object[] { "testEmail@gmail.com", "testEmail@gmail.com" }
    };
}

```

[Theory]

```

[MemberData(nameof(SetNormalGmailEmails))]
public void SetNormalGmail(string email, string gmail)
{
    var newEmail = EmailValidator.SetNormalGmail(email);

    Assert.Equal(email, newEmail);
}

```

[Theory]

```

[MemberData(nameof(InvalidEmails))]
[MemberData(nameof(InvalidDomains))]
public void InvalidSetNormalGmail(string email)
{
    Assert.Throws<FormatException>(() => EmailValidator.SetNormalGmail(email));
}

```

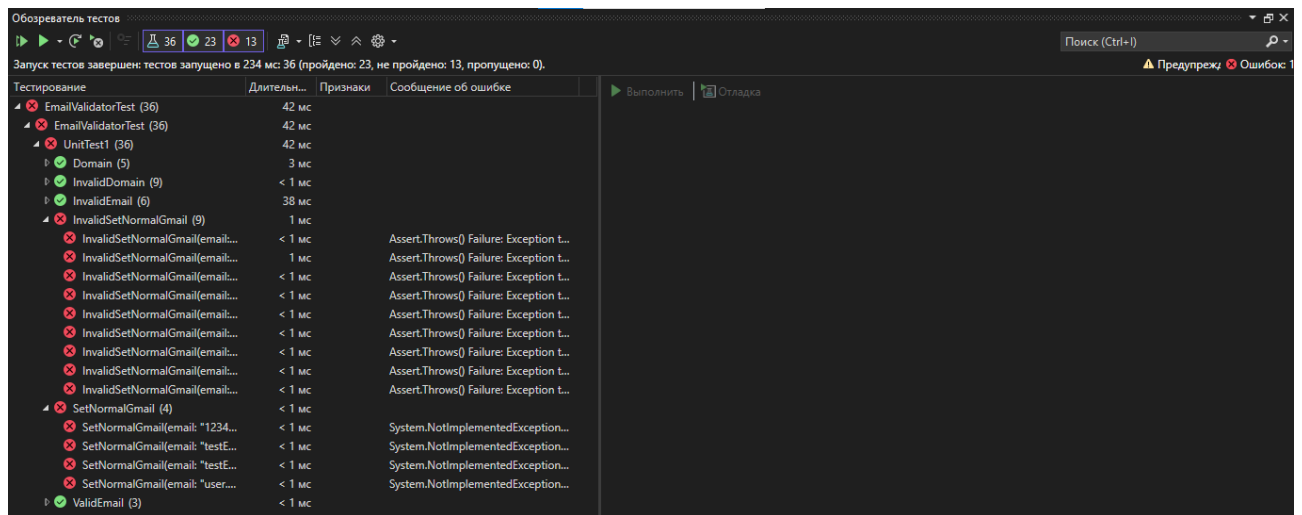


Рисунок 10 – Запущено розролені тести

Здійснено мінімальнк реалізацію для проходження тестів  
`public static string SetNormalGmail(string email)`

```
{
    if (string.IsNullOrEmpty(email) || !IsValidEmail(email))
        throw new FormatException("Невірний формат електронної адреси");
    string userPart = email.Substring(0, email.IndexOf('@'));
    userPart = userPart.Replace(".", "");
    int indexPlus = userPart.IndexOf('+');
    if (indexPlus >= 0)
        userPart = userPart.Substring(0, indexPlus);
    return $"{userPart.ToLower()}@gmail.com";
}
```

Запущено тести для перевірки реалізації

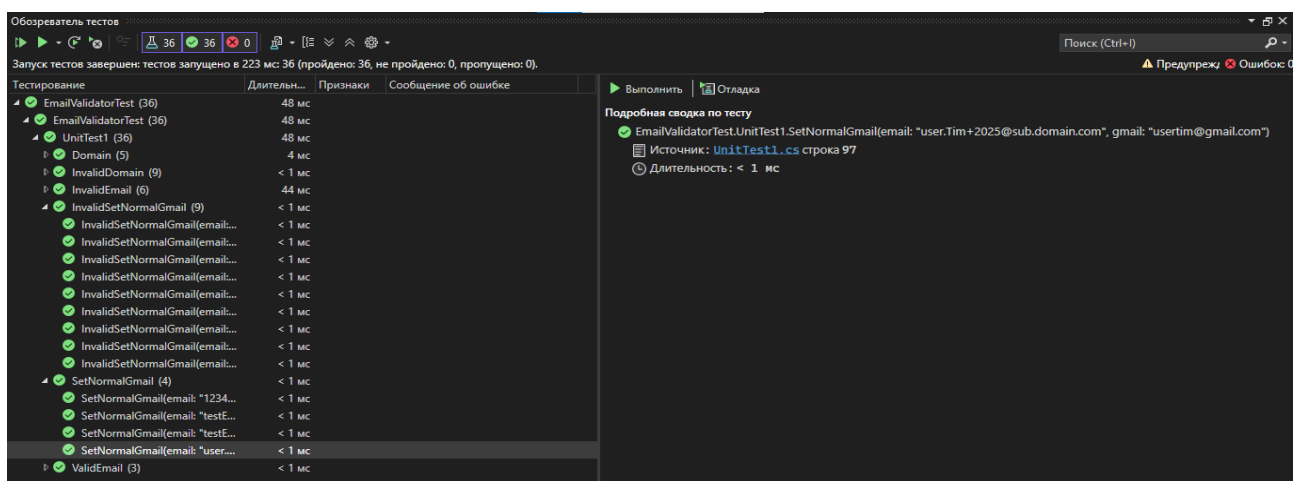


Рисунок 11 – Результат запуску тестів після мінімальної реалізації

Здійснено рефакторинг методу  
`public static string SetNormalGmail(string email)`

```
{
```

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
|       |      |          |        |      |                        | 27   |
| Змін. | Арк. | № докум. | Підпис | Дата |                        |      |

```

ValidateEmail(email);

string userPart = email.Substring(0, email.IndexOf('@'));

userPart = userPart.Replace(".", "");

int indexPlus = userPart.IndexOf('+');

if (indexPlus >= 0)

    userPart = userPart.Substring(0, indexPlus);

return $" {userPart.ToLower()}@gmail.com";

}

private static void ValidateEmail(string email)

{

    if (string.IsNullOrEmpty(email) || !IsValidEmail(email))

        throw new FormatException("Невірний формат електронної адреси");

}

```

Запущено тест на перевірку після рефакторингу:

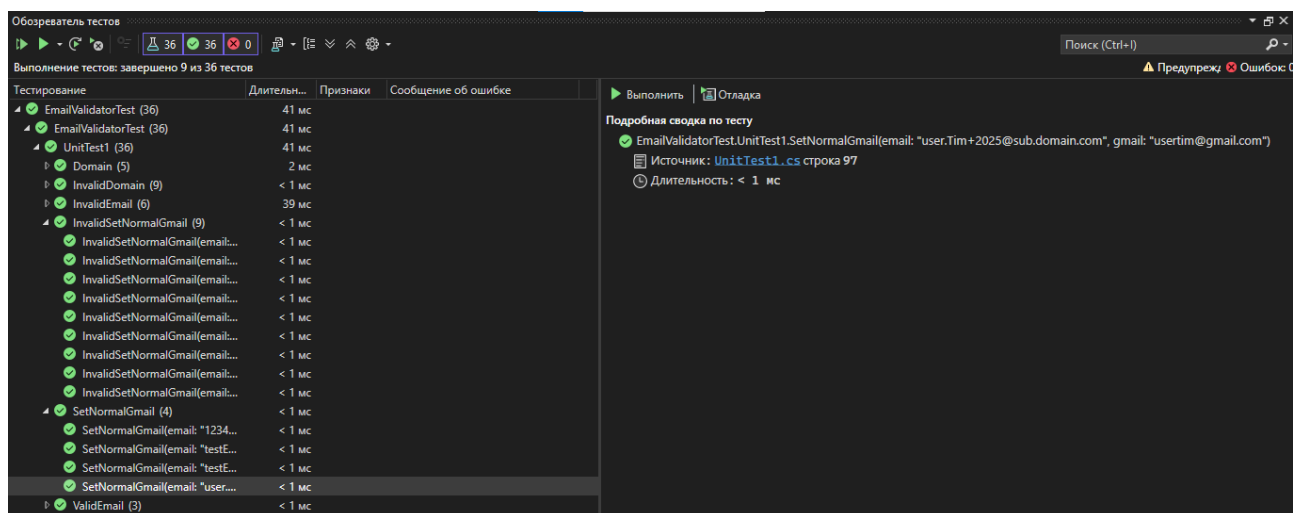


Рисунок 12 – Результат тестів після рефакторингу

#### Завдання 4

Створено CustomWebApplicationFactory, що замінює віддалено БД на локальну із встановленими елементами в SeedPeople:

```

using Microsoft.AspNetCore.Mvc.Testing;

using System;

using System.Collections.Generic;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

using Mongo2Go;

```

```

using Microsoft.AspNetCore.Hosting;
using MongoDB.Driver;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.AspNetCore.Mvc.Authorization;
namespace IntegrationTest.Setting
{
    public class CustomWebApplicationFactory : WebApplicationFactory<Program>
    {
        private MongoDBRunner runner;
        protected override void ConfigureWebHost(IWebHostBuilder builder)
        {
            builder.ConfigureServices(service =>
            {
                runner = MongoDBRunner.Start();
                var descriptor = service.SingleOrDefault(s => s.ServiceType == typeof(IMongoClient
));
                if (descriptor is not null)
                    service.Remove(descriptor);
                service.AddSingleton<IMongoClient>(sp => new MongoClient(runner.ConnectionStr
ing));

                service.AddSingleton(sp =>
                {
                    var Client = sp.GetRequiredService<IMongoClient>();
                    return Client.GetDatabase("IntegrateDB");
                });
                service.AddSingleton<SeedPeople>();
            });
        }
        protected override void Dispose(bool disposing)
        {
            base.Dispose(disposing);
            runner?.Dispose();
        }
    }
}

```

```

    }
}
using LW4_task_3.Models.Entities;
using MongoDB.Driver;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace IntegrationTest.Setting
{
    public class SeedPeople
    {
        private readonly IMongoDatabase _db;
        public SeedPeople(IMongoDatabase db)
        {
            _db = db;
        }
        public async Task SeedPeoplesAsync()
        {
            var collection = _db.GetCollection<PeopleItem>("Peoples");
            await collection.DeleteManyAsync(FilterDefinition<PeopleItem>.Empty);
            var peoples = new List<PeopleItem>()
            {
                new PeopleItem() {Id = "68f5e054f21b02f6aece46ee", Name = "Kate", Email = "Kate@Email.com" },
                new PeopleItem() {Id = "68f5e06bf21b02f6aece46ef", Name = "John", Email = "John@Email.com" },
                new PeopleItem() {Id = "68ff7822a866d7d74955a11c", Name = "Jane", Email = "Jane@Email.com" },
                new PeopleItem() {Id = "690113f4b26a5f31097a0fad", Name = "Steve", Email = "Steve@Email.com" },
                new PeopleItem() {Id = "69039a91e402e8cfbe2a51e1", Name = "Alex", Email = "Alex@Email.com" }
            };
        }
    }
}

```

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
|       |      |          |        |      |                        | 30   |
| Змін. | Арк. | № докум. | Підпис | Дата |                        |      |

```

        await collection.InsertManyAsync(peoples);
    }
}

```

Створено клас BaseIntegration, який встановлює HttpClient на всі тестові класи встановлює вміст локальної БД:

```

using Microsoft.Extensions.DependencyInjection;
using System;
using System.Collections.Generic;
using System.Formats.Tar;
using System.Linq;
using System.Runtime.Serialization;
using System.Text;
using System.Threading.Tasks;
namespace IntegrationTest.Setting
{
    public class BaseIntegration: IClassFixture<CustomWebApplicationFactory>
    {
        protected readonly HttpClient httpClient;
        protected readonly IServiceProvider serviceProvider;
        public BaseIntegration(CustomWebApplicationFactory factory)
        {
            httpClient = factory.CreateClient();
            serviceProvider = factory.Services;
            RunSeedAsync().Wait();
        }
        private async Task RunSeedAsync()
        {
            using var scope = serviceProvider.CreateScope();
            var seeder = scope.ServiceProvider.GetRequiredService<SeedPeople>();
            await seeder.SeedPeoplesAsync();
        }
    }
}

```

Розроблено інтеграційні тести до контролера по отриманню користувачів програми:

```
using IntegrationTest.Setting;
using LW4_task_3.Models.Request;
using LW4_task_3.Models.Response;
using Microsoft.AspNetCore.Http;
using MongoDB.Bson.Serialization.Serializers;
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
namespace IntegrationTest
{
    public class PeopleController : BaseIntegration
    {
        public PeopleController(CustomWebApplicationFactory factory) : base(factory) {}

        [Fact]
        public async Task GetAllPeople_ReturnSeedItems()
        {
            var response = await httpClient.GetAsync("/People");
            response.EnsureSuccessStatusCode();
            var items = await response.Content.ReadFromJsonAsync<List<PeopleResponse>>();
            Assert.NotNull(items);
            Assert.Equal(5, items.Count);
            Assert.Contains(items, item => item.Name == "Kate");
        }

        [Fact]
        public async Task GetById_ReturnPeople()
        {
            var peoples = await httpClient.GetFromJsonAsync<List<PeopleResponse>>("/People");
            string id = peoples[0].Id;
            var response = await httpClient.GetAsync($" /People/{id}");
            response.EnsureSuccessStatusCode();
            var responsePeople = await response.Content.ReadFromJsonAsync<PeopleResponse>();
            Assert.NotNull(responsePeople);
        }
    }
}
```



```

        Assert.Equal(id, responsePeople.Id);
    }
    [Fact]
    public async Task GetById_InvalidId_ReturnNotFound()
    {
        var response = await httpClient.GetAsync("/People/11111");
        Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
    }
    [Fact]
    public async Task Create_ReturnsCreatedItem()
    {
        var newPeople = new PeopleRequest
        {
            Name = "Newpeople",
            Email = "NewEmail@gmail.com"
        };
        var response = await httpClient.PostAsJsonAsync("/People", newPeople);
        Assert.Equal(HttpStatusCode.Created, response.StatusCode);
        var people = await response.Content.ReadFromJsonAsync<PeopleResponse>();
        Assert.NotNull(people);
        Assert.Equal(newPeople.Name, people.Name);
        Assert.Equal(newPeople.Email, people.Email);
        Assert.False(string.IsNullOrEmpty(people.Id));
        var list = await httpClient.GetFromJsonAsync<List<PeopleResponse>>("/People");
        Assert.Equal(list.Count, 6);
    }
    [Fact]
    public async Task Update_ReturnNoContent_UpdateDB()
    {
        var peoples = await httpClient.GetFromJsonAsync<List<PeopleResponse>>("/People");
        string id = peoples[0].Id;
        var updatedPeople = new PeopleRequest
        {

```

```

        Name = "Updatename",
        Email = "UpdateEmail@gmail.com"
    };
    var response = await httpClient.PutAsJsonAsync($"/People/{id}", updatedPeople);
    Assert.Equal(HttpStatusCode.NoContent, response.StatusCode);
    var p = await httpClient.GetFromJsonAsync<PeopleResponse>($"/People/{id}");
    Assert.Equal(updatedPeople.Name, p.Name);
    Assert.Equal(updatedPeople.Email, p.Email);
}
[Fact]
public async Task Update_InvalidId_ReturnNotFound()
{
    var response = await httpClient.PutAsJsonAsync(
        "/People/68f5e054f21b02f6aece46e5",
        new PeopleRequest() { Name = "Somename", Email = "SomeEmail@gmail.com"
    });
    Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
}
[Fact]
public async Task UpdatePart_ReturnNoContent_UpdateDB()
{
    var peoples = await httpClient.GetFromJsonAsync<List<PeopleResponse>>("/People");
    string id = peoples[0].Id;
    string jsonPatch = @"{"name": "Patchname"}";
    var doc = JsonDocument.Parse(jsonPatch);
    var element = doc.RootElement;
    var response = await httpClient.PatchAsJsonAsync($"/People/{id}", element);
    Assert.Equal(HttpStatusCode.NoContent, response.StatusCode);
    var pathedPeople = await httpClient.GetFromJsonAsync<PeopleResponse>($"/People/{id}");
    Assert.Equal(peoples[0].Email, pathedPeople.Email);
    Assert.Equal("Patchname", pathedPeople.Name);
}
[Fact]

```

```

public async Task UpdatePart_InvalidId_ReturnNotFound()
{
    string jsonPatch = @"{"name": "Patchname"}";
    var doc = JsonDocument.Parse(jsonPatch);
    var element = doc.RootElement;

    var response = await httpClient.PatchAsJsonAsync(
        "/People/68f5e054f21b02f6aece46e5",
        element
    );

    Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
}
[Fact]

```

```

public async Task DeletePeople_ReturnNoContent_CheckDB()
{
    var list = await httpClient.GetFromJsonAsync<List<PeopleResponse>>("/People");
    string id = list[0].Id;

    var response = await httpClient.DeleteAsync($"People/{id}");
    Assert.Equal(HttpStatusCode.NoContent, response.StatusCode);

    list = await httpClient.GetFromJsonAsync<List<PeopleResponse>>("/People");
    Assert.Equal(4, list.Count);
}
[Fact]

```

```

public async Task Delete_InvalidId_NotFound()
{
    var response = await httpClient.DeleteAsync("/People/68f5e054f21b02f6aece46e5");
    Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
}
}
}

```

Виконано інтеграційні тести:

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
|       |      |          |        |      |                        | 35   |
| Змін. | Арк. | № докум. | Підпис | Дата |                        |      |

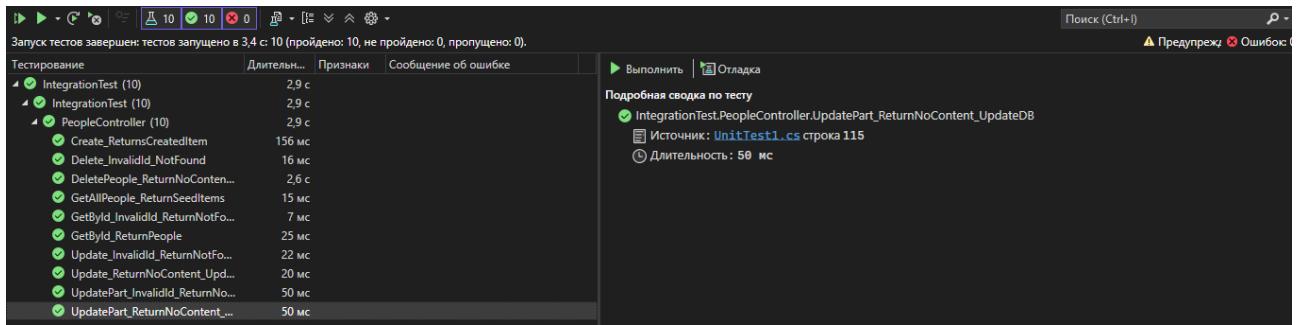


Рисунок 13 – Виконані інтеграційні тести

## ВИСНОВОК:

На даній лабораторній роботі я навчився розробляти Unit – тести до вже готових класів, а також розробляти Unit – тести за технологією TDD. Також навчився реалізовувати інтеграційні тести, задопоогою заміни віддаленої БД

|       |      |          |        |      |                        |      |
|-------|------|----------|--------|------|------------------------|------|
|       |      |          |        |      | ЛР.ОК24.ПІ231.03.07-08 | Арк. |
| Змін. | Арк. | № докум. | Підпис | Дата |                        | 36   |